

Lab Assignment: Smart Delivery Cost Calculator

Submission Details

- **Submission Platform:** Google Classroom
- **Submit:**
 - Your Java file (`DeliveryCalculator.java`)
 - At least 2 screenshots showing program execution

Objective

The goal of this assignment is to develop a Java program that calculates delivery costs intelligently. Students will practice the following concepts:

- Declaring Methods with Multiple Parameters
- Method Overloading
- Enum Usage
- Conditional Logic (if/else)
- User Input using Scanner

Program Description

In this assignment, you will create a program that calculates the **total delivery cost** based on different parameters such as weight, distance, delivery type, and customer type.

Program Requirements

User Input

- The program must ask the user to enter:
 - Weight (double)
 - Distance (double)
 - Express delivery (boolean)
 - Customer type (String)
- The program must validate:
 - Weight cannot be negative
 - Distance must be positive
 - Customer type must be **regular** or **premium**
- If the input is invalid, the program must repeatedly ask for input.

Enum Requirement

Define the following enum:

```
1 enum CustomerType {  
2     REGULAR,  
3     PREMIUM  
4 }
```

Method Overloading Requirement

You must implement the following overloaded methods:

```
1 calculateCost(double weight, double distance)  
2 calculateCost(double weight, double distance, boolean isExpress)  
3 calculateCost(double weight, double distance, boolean isExpress,  
    ↪ CustomerType customerType)
```

Note: All calculations must be performed in the most detailed method.

Cost Calculation Rules

Base Cost (Weight):

- 0–5 kg → 10 TL
- 5–20 kg → 20 TL
- 20+ kg → 40 TL

Distance Cost:

- First 50 km → free
- Additional distance → 0.5 TL per km

Express Delivery:

- If express → increase total cost by 50%

Customer Type:

- PREMIUM → 20% discount
- REGULAR → no discount

Important:

- If both express and premium are selected:
 - First apply express increase
 - Then apply discount

Helper Method Requirement

Implement the following method:

```
1 parseCustomerType(String input)
```

- Converts String input to CustomerType
- Returns null if input is invalid

Additional Task (Advanced)

Implement the following method:

```
1 findCheapestOption(double weight, double distance)
```

- The method must calculate:
 - Normal + Regular
 - Express + Regular
 - Normal + Premium
 - Express + Premium
- It must return the **cheapest option** as a String.

Sample Input

```
1 Enter weight (kg): 10
2 Enter distance (km): 120
3 Is express delivery? (true/false): true
4 Enter customer type (regular/premium): premium
```

Sample Output

```
1 Total Cost: 54.0 TL
2 Best Option: Normal Delivery for Premium Customer
```

Technical Requirements

- Use **method overloading**
- Use **enum**
- Use **Scanner** for input
- Use **if/else** statements
- Avoid code repetition
- Write clean and readable code